

# Branch 사용법

## git switch -c - 브랜치 생성

- 새 브랜치를 만들 때, 현재 브랜치의 상태를 기준으로 만들기 때문에 현재 브랜치가 무엇인지 꼭 확인해야 합니다.

## 작업

1. 브랜치 확인
2. 새로운 브랜치(add-color) 생성

```
$ git status # 1.  
$ git switch -c add-color # 2.
```

```
good593 👑 ~/dev/sample ➤ master
> git status
현재 브랜치 master
커밋할 사항 없음, 작업 폴더 깨끗함
good593 👑 ~/dev/sample ➤ master
> git switch -c add-color
새로 만든 'add-color' 브랜치로 전환합니다
good593 👑 ~/dev/sample ➤ add-color
> git status
현재 브랜치 add-color
커밋할 사항 없음, 작업 폴더 깨끗함
good593 👑 ~/dev/sample ➤ add-color
> 
```

git switch의 `-c` 옵션은 브랜치 생성과 브랜치 이동을 한번에 수행합니다. 다음 명령어와 동일합니다.

```
git branch add-color
git swtich add-color
```

sh

# add-color 브랜치에서 작업

## 작업

1. green, blue 파일 추가
2. 전체 변경사항을 인덱스에 추가
3. 커밋 작성

```
$ touch green blue # 1.  
$ echo "green1" >> green  
$ echo "blue1" >> blue  
$ git add -A # 2.  
$ git commit -m "add green, blue" # 3.
```

```
good593 👑 ~/dev/sample ➤ add-color
➤ touch green blue
good593 👑 ~/dev/sample ➤ add-color
➤ echo "green1" >> green
good593 👑 ~/dev/sample ➤ add-color
➤ echo "blue1" >> blue
good593 👑 ~/dev/sample ➤ add-color
➤ git add -A
good593 👑 ~/dev/sample ➤ add-color +
➤ git commit -m "add green, blue"
[add-color d59b543] add green, blue
2 files changed, 2 insertions(+)
create mode 100644 blue
create mode 100644 green
good593 👑 ~/dev/sample ➤ add-color
➤
```

## git switch - 브랜치 변경

add-color 브랜치에서 진행한 작업이 기존 main 브랜치와 독립적인 것을 확인합니다.

### 작업

1. 현재(add-color) 파일들 확인
2. 브랜치 리스트 확인
3. master 브랜치로 이동
4. 파일들 다시 확인
  - green, blue 파일이 없음!!

```
$ ls # 1.  
$ git branch # 2. : 나가기; q  
$ git switch master # 3.  
$ ls # 4.
```

```
good593 👑 > ~/dev/sample > ? add-color  
▶ ls  
blue green orange red yellow  
good593 👑 > ~/dev/sample > ? add-color  
▶ git branch  
good593 👑 > ~/dev/sample > ? add-color  
▶ git switch master  
'master' 브랜치로 전환합니다  
good593 👑 > ~/dev/sample > ? master  
▶ ls  
orange red yellow  
good593 👑 > ~/dev/sample > ? master  
▶ □
```

# update-red 브랜치 추가

## 작업

1. update-red 브랜치 생성 후 이동
2. red 파일 내용 변경
3. 전체 변경사항을 인덱스에 추가
4. 커밋 작성

```
$ git switch -c update-red # 1.  
$ echo "red11" >> red # 2.  
$ git add -A # 3.  
$ git commit -m "update red" # 4.
```

```
good593 👑 ~/dev/sample ➤ master  
➤ git switch -c update-red  
새로 만든 'update-red' 브랜치로 전환합니다  
good593 👑 ~/dev/sample ➤ update-red  
➤ echo "red11" >> red  
good593 👑 ~/dev/sample ➤ update-red ±  
➤ git add -A  
good593 👑 ~/dev/sample ➤ update-red +  
➤ git commit -m "update red"  
[update-red 3cbe9f4] update red  
1 file changed, 1 insertion(+)  
good593 👑 ~/dev/sample ➤ update-red  
➤
```

## 현재 브랜치 상황

- master: red, orange, yellow
- add-color: red, orange, yellow, green, blue
- update-red: red(변경), orange, yellow

## git merge - 브랜치 합치기

### 작업

1. master 브랜치로 이동
2. add-color 브랜치의 수정사항을 master 브랜치로 merge(합치기)
3. add-color에서 작성한 커밋 로그가 master 브랜치에도 추가된 것을 확인
4. master 브랜치에서 green, blue 파일이 추가된 것을 확인

```
$ git switch master # 1.  
$ git merge add-color # 2.  
$ git log # 3.  
$ ls # 4.
```

```
good593 👑 > ~/dev/sample > update-red  
▶ git switch master  
'master' 브랜치로 전환합니다  
good593 👑 > ~/dev/sample > master  
▶ git merge add-color  
업데이트 중 1a1beb6..d59b543  
Fast-forward  
blue | 1 +  
green | 1 +  
2 files changed, 2 insertions(+)  
create mode 100644 blue  
create mode 100644 green  
good593 👑 > ~/dev/sample > master  
▶ git log  
good593 👑 > ~/dev/sample > master  
▶ ls  
blue green orange red yellow  
good593 👑 > ~/dev/sample > master  
▶
```

```
commit d59b54395dd7accb472fbef975e62bd213f8c56f (HEAD -> master, add-color)
```

```
Author: good593 <goodwon593@gmail.com>
```

```
Date: Wed Dec 20 11:03:43 2023 +0900
```

```
add green, blue
```

```
commit 1a1beb6cc21a1d5efaa092be3b66668b1afad296
```

```
Author: good593 <goodwon593@gmail.com>
```

```
Date: Wed Dec 20 10:30:14 2023 +0900
```

```
new revert
```

```
commit e823044d4c1959b047f25b54ad777dfafba6debf
```

```
Author: good593 <goodwon593@gmail.com>
```

```
Date: Wed Dec 20 10:22:48 2023 +0900
```

```
v4 commit
```

```
commit 4bc138dd66f51052a68aa217d510cdf8bac974ac
```

```
Author: good593 <goodwon593@gmail.com>
```

```
Date: Wed Dec 20 10:12:08 2023 +0900
```

```
v2 commit
```

```
:|
```

## 결과 정리

- add-color에서 작업한 내용(파일들)이 master 브랜치로 머지됨
- master 브랜치에 green, blue 파일이 추가된 것을 확인
- add-color에서 작성한 커밋 로그가 master 브랜치에도 추가된 것을 확인

## conflict - 충돌 해결

update-red 브랜치에서 수정한 red 파일을 master 브랜치에서도 수정하여 충돌 상황을 만듭니다.

### 작업

1. master 브랜치에서 red 파일 수정
2. 전체 변경사항을 인덱스에 추가
3. 커밋 작성
4. update-red 브랜치를 master 브랜치로 머지(합치기)
  - 충돌발생!!

```
$ echo "red111" >> red
$ git add -A
$ git commit -m "update red111"
$ git merge update-red # 충돌발생!!!!
```

```
good593 👑 > ~/dev/sample > master
▶ echo "red111" >> red
good593 👑 > ~/dev/sample > master ±
▶ git add -A
good593 👑 > ~/dev/sample > master +
▶ git commit -m "update red111"
[master b413484] update red111
1 file changed, 1 insertion(+)
good593 👑 > ~/dev/sample > master
▶ git merge update-red
자동 병합: red
충돌 (내용): red에 병합 충돌
자동 병합이 실패했습니다. 충돌을 바로잡고 결과물을 커밋하십시오.
✖ good593 👑 > ~/dev/sample > master ±+ >M<
▶ █
```

## 충돌내용 확인

- red 파일 확인(충돌내용 확인!!)

```
$ cat red # 충돌내용 확인!!
```

```
good593 👑 > ~/dev/sample > master ±+ >M<
> cat red
빨강
<<<<<<< HEAD
red111
=====
red11
>>>>>>> update-red
good593 👑 > ~/dev/sample > master ±+ >M<
> □
```

## 충돌 해결

1. red 파일에서 필요없는 내용 모두 삭제(수정)
  2. 전체 변경사항을 인덱스에 추가
  3. 커밋 진행(메세지는 생략)
- vi창이 열리면, `ecs` 키를 누르고 `:x` 를 입력한 후 엔터

```
$ git add -A  
$ git commit
```

```
good593 👑 ~/dev/sample ➤ master ±+ >M<  
➤ cat red  
빨강  
good593 👑 ~/dev/sample ➤ master ±+ >M<  
➤ git add -A  
good593 👑 ~/dev/sample ➤ master + >M<  
➤ git commit  
[master 506f0ef] Merge branch 'update-red'  
good593 👑 ~/dev/sample ➤ master
```